Introduction to MERN & React JS Foundations

Full Stack Development – Session 1

🎯 Today's Focus

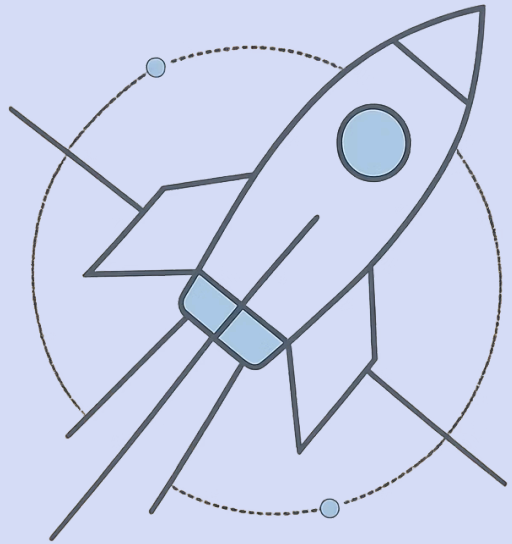
React fundamentals + environment setup

🛠️ Hands-On Goal

Build and run your first React application

🗺️ The Journey

MERN architecture from ground up



Made with GAMMA

Expected Learning Outcomes

By the end of this session, you will be able to:

1

Understand MERN Architecture

Grasp how Full Stack apps connect frontend, backend, and database

2

Learn Core React Concepts

Components, JSX, props, state, and the Virtual DOM

3

Set Up Your Dev Environment

Install Node.js, npm, and VS Code — ready to code

4

Build Your First React App

Scaffold, run, and explore a real React project

Made with GAMMA

Full Stack Overview

A Full Stack application combines three essential layers — each playing a distinct role in delivering a complete web experience.

1

Frontend

User Interface built with **React.js** — what users see and interact with

2

Backend

Server logic powered by **Node.js** and **Express.js** — handles requests and APIs

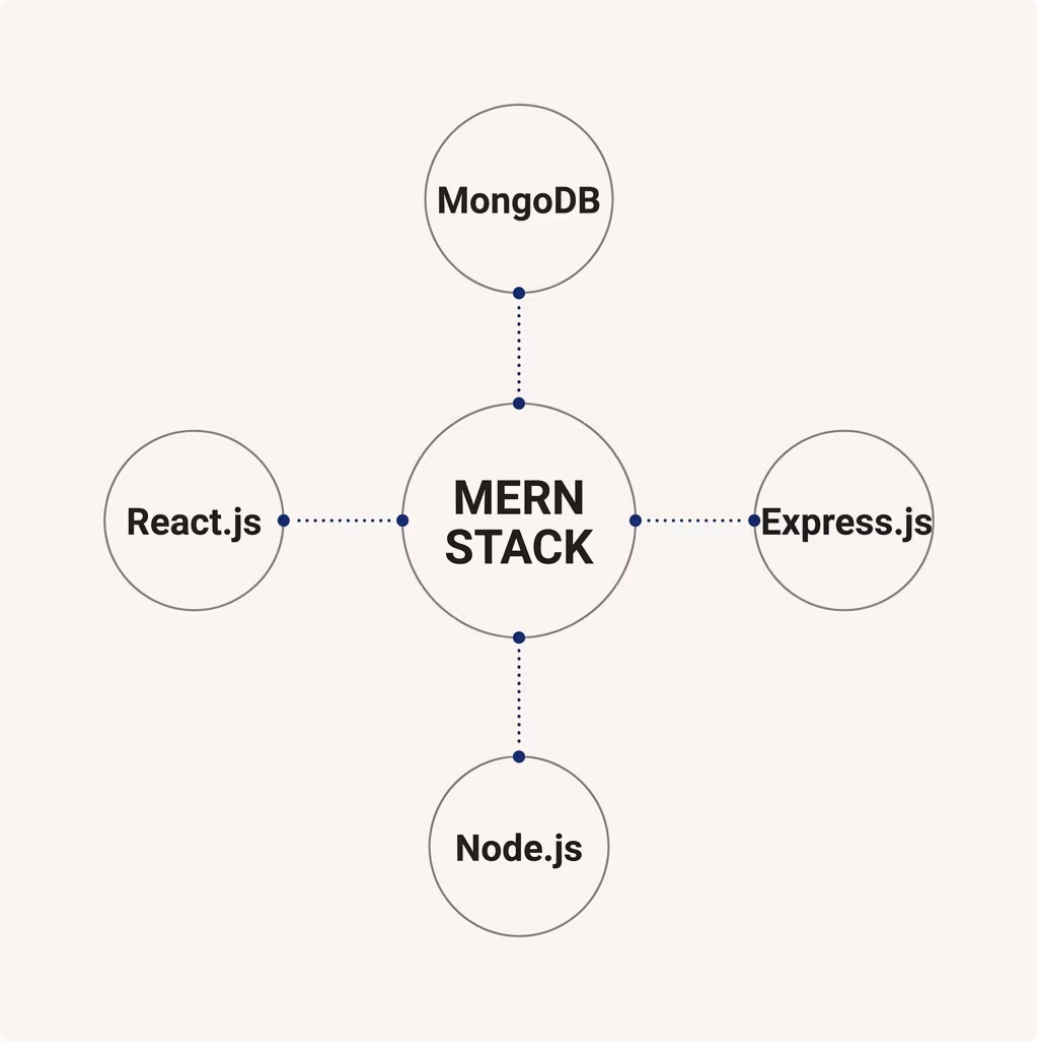
3

Database

Data stored and retrieved via **MongoDB** — the persistence layer

Made with GAMMA

MERN Architecture



M — MongoDB

NoSQL database storing flexible, JSON-like documents

E — Express.js

Lightweight backend framework for routing and APIs

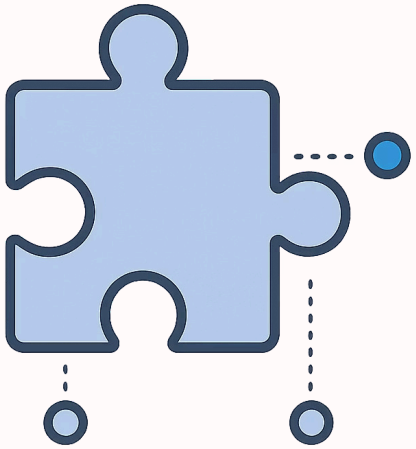
R — React.js

Frontend library for building dynamic user interfaces

N — Node.js

JavaScript runtime that powers the server side

Made with GAMMA



SESSION 1

What is React?

React is an open-source **JavaScript library** for building fast, interactive user interfaces — originally developed by **Facebook** in 2013.

🧩 Component-Based

Build encapsulated, reusable UI pieces

⚡ Dynamic UIs

Pages update without full reloads

🌍 Industry Standard

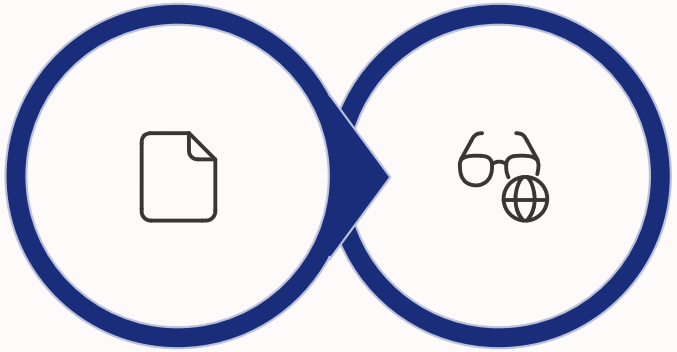
Used by Meta, Netflix, Airbnb & more

Made with GAMMA

Core Concepts: SPA & Virtual DOM

Single Page Application (SPA)

React apps load a **single HTML page**. Instead of fetching a new page from the server on every click, React dynamically updates only the content that changes — resulting in a smoother, app-like experience.



Page Loads Once

Content Updates Dynamicall y

DOM vs. Virtual DOM

Real DOM

The actual page structure — expensive to update because the whole tree re-renders

Virtual DOM

A lightweight in-memory copy — React diffs changes and updates only what's necessary, making it **significantly faster**

Made with GAMMA

React Pros & Cons

React is a powerful tool — but like any technology, it comes with trade-offs worth knowing upfront.

✅ Advantages

→ Fast Rendering

Virtual DOM ensures only changed elements are updated

→ Reusable Components

Write once, use across your entire application

→ Strong Ecosystem

Massive community, rich libraries, and extensive documentation

⚠️ Limitations

→ Learning Curve

JSX syntax and state management can be tricky for beginners

→ UI Layer Only

React handles the frontend — you still need Node, Express, and MongoDB for a full app

→ Frequent Updates

The ecosystem evolves quickly; staying current requires effort

Made with GAMMA

Development Environment Setup

Before writing a single line of React, get your machine ready with these three essential tools.

Node.js

The JavaScript runtime that powers your backend and tooling. Download from **nodejs.org** and verify with:

```
node -v
npm -v
```

npm

Node Package Manager — bundled with Node.js. Used to install React and all project dependencies. No separate install needed.

Visual Studio Code

The recommended code editor. Download from **code.visualstudio.com**. Install the *ES7+ React Snippets* extension for productivity.

Made with GAMMA

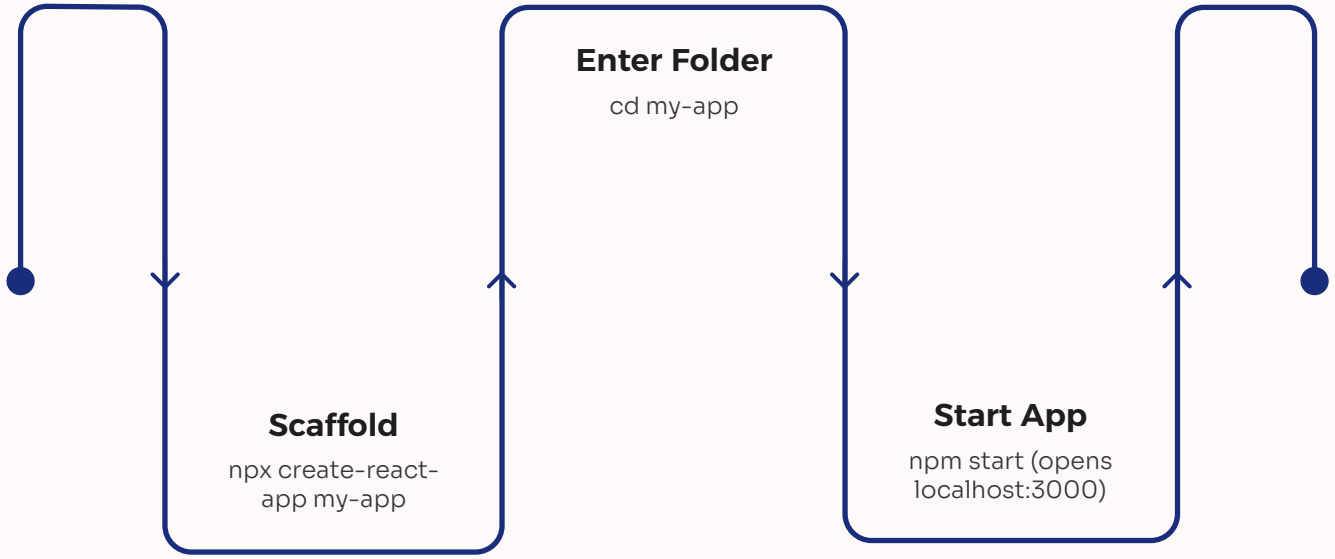
Create Your First React App

Three Commands to Get Started

React's official scaffolding tool — **Create React App** — sets up everything automatically. Run these commands in your terminal:

```
npx create-react-app my-app
cd my-app
npm start
```

Your default React app will open automatically at **http://localhost:3000** in the browser. No configuration needed!



Made with GAMMA

React Project Structure & Core Concepts

📁 Folder Structure

src/

Your main application code — components, logic, styles

public/

Static files like `index.html` — the single page React mounts into

App.js

The root component — everything renders through here

Flow: **index.js** → mounts **App.js** → renders your **UI components**

⚛️ React Building Blocks

JSX

HTML-like syntax written inside JavaScript files

Components

Functional or Class-based reusable UI blocks

Props

Pass data into a component from its parent

State

Internal data that drives dynamic UI changes

Events

Handle clicks, inputs, and user interactions

Made with GAMMA